

**Skills**  
Network

::page{title="Reading: Monitoring and Optimizing Your Databases in MySQL"}

**Estimated time needed:** 25 minutes

In this reading, you'll learn how you can monitor and optimize your queries with the help of the tool, phpMyAdmin.

## Objectives

After completing this reading, you will be able to:

1. Identify the purpose of monitoring and optimizing your databases
2. Describe the steps to monitoring the health of your database with phpMyAdmin
3. Apply the best practices in optimizing your database

## Software Used

In this reading, you will see usage of [MySQL](#). MySQL is a Relational Database Management System (RDBMS) designed to efficiently store, manipulate, and retrieve data.



## Monitor Your Database

Database monitoring refers to the tasks related to reviewing the operational status of your database. Monitoring is critical in maintaining the health and performance of your database and with proper monitoring, any problems that appear can be identified and resolved in a timely manner. This will keep your database running and avoid any outages that may affect yourself or your end users.

Tools such as phpMyAdmin, an open-source administration tool for MySQL and MariaDB, have built-in features that help with monitoring your database. Particularly in phpMyAdmin, you can gain a better understanding of how your server and its databases are functioning through the **Status** page.

Databases

SQL

Status

Server

Processes

Query statistics

Network traffic since startup: 10.3 MiB

This MySQL server has been running for 0 days, 0 h

| Traffic  | #        | Ø per hour | Connections         |
|-------------------------------------------------------------------------------------------|----------|------------|---------------------|
| Received                                                                                  | 1.7 MiB  | 5.7 MiB    | Max. connections    |
| Sent                                                                                      | 8.7 MiB  | 29.2 MiB   | Failed attempts     |
| Total                                                                                     | 10.3 MiB | 34.9 MiB   | Aborted connections |
|                                                                                           |          |            | Total               |

This MySQL server works as master in replication

## Replication status

### Master status

| Variable         | Value         |
|------------------|---------------|
| File             | binlog.000002 |
| Position         | 1690985       |
| Binlog_Do_DB     |               |
| Binlog_Ignore_DB |               |

### Server Status

On the Server page, you're provided with more information about the server status, including how long you've been running the server for.

1. The **Traffic** table shows the incoming and outgoing traffic, where **Received** is the number of bytes received by the server (incoming), and **Sent** is then number of bytes sent out by the server (outgoing). These numbers can tell you how much traffic you're receiving and based on performance, whether you're receiving unusually high traffic.

2. The **Connections** table gives you information about the maximum number of connections you can have at the same time, the number of failed connection attempts, the number of aborted connection attempts, and the total number of connections (successful or not).

## Network traffic since startup: 10.3 MiB

This MySQL server has been running for 0 days, 0 ho

| Traffic  | #        | Ø per hour | Connections |
|-------------------------------------------------------------------------------------------|----------|------------|-------------|
| Received                                                                                  | 1.7 MiB  | 5.7 MiB    | Max. conc   |
| Sent                                                                                      | 8.7 MiB  | 29.2 MiB   | Failed atte |
| Total                                                                                     | 10.3 MiB | 34.9 MiB   | Aborted     |
|                                                                                           |          |            | Total       |

## Processes

On the Processes page, you can see the operations currently being performed within the server.

There will always be at least two users: **event\_scheduler** and **root**.

Server

Processes

Query statistics

A

Filters

☐ Show only active

Refresh

| Processes            | ID   | User            | Host            |
|----------------------|------|-----------------|-----------------|
| <a href="#">Kill</a> | 5    | event_scheduler | localhost       |
| <a href="#">Kill</a> | 1975 | root            | 172.25.0.3:4020 |
| <a href="#">Kill</a> | 1976 | root            | 172.25.0.3:4020 |


**Note:** Enabling the auto refresh here might cause

Refresh rate:

5 seconds ▼

▶ Start auto refresh

This table shows the **ID** of the process; the **User** associated with it, the **Host** to which it's connected to; the default **Database** or None; the **Command** type, if it's being executed or if the session is idle (Sleep); the **Time** it has been in its **Status** for; and the **Status** which gives a description of what it's doing right now.

As can be seen in the previous table, you can replicate the displayed process list in the command-line interface with the following command:

```
SHOW PROCESSLIST
```

```
mysql> SHOW PROCESSLIST;
+-----+-----+-----+
| Id      | User                | Host                |
+-----+-----+-----+
|      5  | event_scheduler     | localhost           |
| 2548    | root                | 172.25.0.1:4873    |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

## Query Statistics

On the Query Statistic page, you'll see the types of queries run and how often they've been run. These include queries that you have run, in addition to queries run by phpMyAdmin in the background.

The page also offers a visualization of the breakdown of usage.

Questions since startup: 19,323 

- ø per hour: 16,500
- ø per minute: 275
- ø per second: 5

| Statements                 | #     | ø per hour | %  |
|----------------------------|-------|------------|---------------------------------------------------------------------------------------|
| <b>select</b>              | 8,110 | 6,925      | 41.97                                                                                 |
| <b>insert</b>              | 5,208 | 4,447.1    | 26.95                                                                                 |
| <b>set option</b>          | 2,388 | 2,039.1    | 12.36                                                                                 |
| <b>show variables</b>      | 748   | 638.7      | 3.87                                                                                  |
| <b>change db</b>           | 741   | 632.7      | 3.83                                                                                  |
| <b>show binlogs</b>        | 551   | 470.5      | 2.85                                                                                  |
| <b>show grants</b>         | 551   | 470.5      | 2.85                                                                                  |
| <b>show status</b>         | 290   | 247.6      | 1.50                                                                                  |
| <b>show master status</b>  | 170   | 145.2      | 0.88                                                                                  |
| <b>show slave status</b>   | 170   | 145.2      | 0.88                                                                                  |
| <b>show replica status</b> | 170   | 145.2      | 0.88                                                                                  |
| <b>show processlist</b>    | 137   | 117        | 0.71                                                                                  |
| <b>create table</b>        | 37    | 31.6       | 0.19                                                                                  |
| <b>show tables</b>         | 25    | 21.3       | 0.13                                                                                  |
| <b>show fields</b>         | 9     | 7.7        | 0.05                                                                                  |
| <b>show warnings</b>       | 5     | 4.3        | 0.03                                                                                  |
| <b>show keys</b>           | 4     | 3.4        | 0.02                                                                                  |
| <b>drop table</b>          | 3     | 2.6        | 0.02                                                                                  |
| <b>create db</b>           | 2     | 1.7        | 0.01                                                                                  |
| <b>show databases</b>      | 2     | 1.7        | 0.01                                                                                  |
| <b>show create table</b>   | 1     | 0.9        | 0.01                                                                                  |

|              |          |            |             |
|--------------|----------|------------|-------------|
| <b>flush</b> | <b>1</b> | <b>0.9</b> | <b>0.01</b> |
|--------------|----------|------------|-------------|

As we can expect, `SELECT` is the command that has been used the most in the previous above. This result makes sense because we are often selecting specific columns and rows from our database to view.

## All Status Variables

On the All Status Variables page, you'll see the variables listed, their value and their description. If the value of your variable is red, then that means it may need to be investigated and adjusted, if necessary.

For example, in this case **Aborted connects**, which is the number of failed attempts to connect to the server, is red. The number is rather high, meaning it should be looked into. A high number of aborted connects could mean that the program was not closed properly, the session timed out, or an error occurred in the middle of a data transfer.

**Server**

**Processes**

**Query statistics**

**A**

**Filters**

Containing the word:

**Variable**

**Aborted clients**

**Aborted connects**

**Acl cache items count**

**Binlog cache disk use**

**Binlog cache use**

**Binlog stmt cache disk use**

**Binlog stmt cache use**

**Bytes received**

**Bytes sent**

**Caching sha2 password rsa public key**

**Com admin commands**

**Com assign to keycache**

## Com alter db

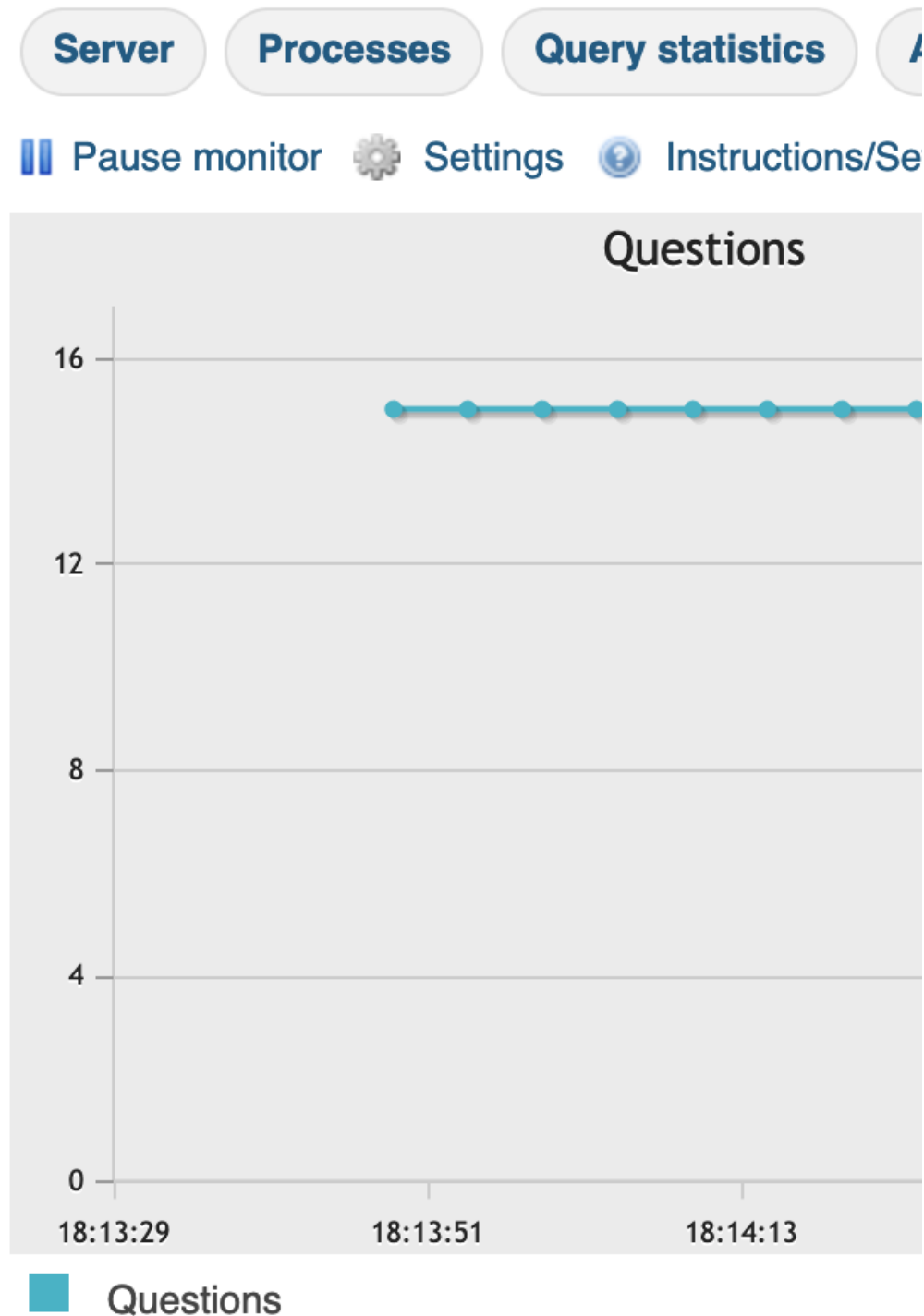
### Com alter event

For more details about each variable and its meaning, you can read the [Server Status Variables Documentation](#) by MySQL.

### Monitor

With the Monitor page, you can optimize your server configuration and identify issues such as intensive queries based on the provided real-time graphs.

By default, you may only see three charts: **Questions**, **Connections / Processes** and **Traffic**.



**Questions** is the number of queries that are being run at a given time, including any background queries by phpMyAdmin. Similar to what is displayed on the **Server** page, the **Connections** /

**Processes** and **Traffic** provide a visual of the current connections, processes and incoming and outgoing bytes received and sent.

These real-time charts can help you spot any irregularities in your databases, such as a sudden spike in querying, connections or traffic. Additional charts can also be added, such as a chart that displays the system's CPU usage or system's memory. These will be further explored in the lab.

## **Advisor**

phpMyAdmin also offers an Advisor system that provides recommendations based on analysis of server variables to help monitor and optimize your database.

 Instructions

## Possible performance issues

| Issue                                                              | Recommendation                                                                   |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Uptime is less than 1 day, performance tuning may not be accurate. | To have more accurate performance tuning, run the system for at least 1 day.     |
| The slow query log is disabled.                                    | Enable slow query log.                                                           |
| There are lots of rows being sorted.                               | While there are many rows, sorting can result in memory issues.                  |
| There are too many joins without indexes.                          | This means that the system has to read a lot of data to find the rows to join.   |
| The rate of reading the first index entry is high.                 | This usually indicates a lot of DELETES, or a lot of updates to the index.       |
| The rate of reading data from a fixed position is high.            | This indicates that the system is reading a lot of data from a fixed position.   |
| The rate of reading the next table row is high.                    | This indicates that the system is reading a lot of data from the next table row. |
| MyISAM key buffer (index cache) % used is low.                     | You may not need a larger key buffer.                                            |
| The rate of opening tables is high.                                | Opening tables can be slow.                                                      |
| The rate of opening files is high.                                 | Consider increasing the number of files that can be opened at once.              |
| Too many table locks were not granted immediately.                 | Optimize the table locks.                                                        |
| Too many connections are aborted.                                  | Connection pool size is too small.                                               |
| Too many connections are aborted.                                  | Connection pool size is too small.                                               |

Please note, the system's recommendations are based on general rules and simple calculations that may not apply to your database, so it's important to understand what you are changing with

the MySQL documentation and how you can undo that change if there was no improvement from it. It's generally best practice to tune your system by changing one setting at a time.

Now that you know how to monitor the health of your database, let's take a look at how we can optimize it!

## Optimize Your Database

Database optimization refers to maximizing the speed and efficiency of retrieving data from your database, improving the experience for both yourself and your learners. Database optimization can also help with improving the performance of slow queries.

It's best to keep your data types simple, using the smallest or a specialized data type when possible. Smaller data types tend to be faster because they require less space, with specialized data types such as date and time (**DATETIME**) optimized to save disk space and memory. To understand the amount of storage a data type can take, you can read the [MySQL Data Types Documentation](#).

In addition, it's helpful to avoid **NULL** values in columns when possible. Non-null values make operations faster as they can use indexes better and eliminate the need to check if each value is **NULL**.

Let's take a look at the various data types and how we can best use them.

### Numeric Data Types

In MySQL, two of the most commonly used data types are: integer types and fixed-point types.

#### Integer Types

With Integer Types, we see that MySQL supports **TINYINT**, **SMALLINT**, **MEDIUMINT**, **INT** and **BIGINT**. Depending on your use, you should choose the smallest possible data type.

From the [Integer Types Documentation](#) page by MySQL, we notice a table similar to the following:

| Type      | Storage (Bytes) | Minimum Value Signed | Maximum Value Signed | Minimum Value Unsigned | Maximum Value Unsigned |
|-----------|-----------------|----------------------|----------------------|------------------------|------------------------|
| TINYINT   | 1               | -128                 | 127                  | 0                      | 255                    |
| SMALLINT  | 2               | -32,768              | 32,767               | 0                      | 65,535                 |
| MEDIUMINT | 3               | -8,388,608           | 8,388,607            | 0                      | 16,777,215             |
| INT       | 4               | -2,147,483,648       | 2,147,483,647        | 0                      | 4,294,967,295          |
| BIGINT    | 8               | -2 <sup>63</sup>     | 2 <sup>63</sup> -1   | 0                      | 2 <sup>64</sup> -1     |

Based on the table presented in the MySQL documentation, we can see that there are six columns **Type**, **Storage (Bytes)**, **Minimum and Maximum Value Signed** and **Minimum and Maximum Value Unsigned**. Unsigned values mean that the values are always non-negative values whereas signed values can be negative, as can be seen in the maximum and minimum values of each type.

This table is telling us how much storage each type uses. For example, **TINYINT** uses the least amount of storage and **BIGINT** uses the most. When looking at their maximum and minimum values, this makes sense because the range of unsigned values for **TINYINT** is 0 to 255, whereas for **BIGINT** it is 0 to a very large number. As such, it'll take up more space.

If you know that your values will only go up to 255, such as if you're logging information of sports teams and there's only 200 in the league, then it would be better to use **TINYINT** over **BIGINT** or **INT**.

#### Fixed-Point Types

Fixed-point types represent non-integer numbers. In MySQL, the two data types are **DECIMAL** and **NUMERIC**, both of which preserve the precision of a number. For example, if you wanted to store monetary data, this data type would come in handy.

## String Data Types

The two most popular types of strings are **CHAR** and **VARCHAR**, and you can optimize these by limiting the number of characters in a column.

**VARCHAR** columns have variable-length strings, meaning in terms of storage, it holds the characters you assign it to. It will also take up an additional two bytes for storage purposes. **CHAR** columns had fixed length, meaning regardless of the actual characters in the column, it will always use the same amount of space.

For example, **CHAR(3)** will always use 3 bytes, even if those bytes are whitespace. The amount of storage **VARCHAR(3)** takes, however, will depend on the contents of that column. Therefore, **CHAR** is a good option if you know that every entry in the column will remain fixed, otherwise, **VARCHAR** may be the better option.

Generally, if you have a value that can be represented by either a string or a number, you should choose the numeric data type. Numeric values take less storage than strings, therefore optimizing your database.

## Date and Time Data Types

MySQL also has special data types for date and time. These types include **DATE**, **TIME**, **DATETIME**, **TIMESTAMP** and **YEAR**.

**DATE** values are displayed as YYYY-MM-DD, **TIME** values are displayed as hh:mm:ss, **DATETIME** and **TIMESTAMP** values are displayed as a combination of both as YYYY-MM-DD hh:mm:ss, and **YEAR** values are displayed as YYYY.

It's best practice to use the appropriate date or time data types rather than, for example, strings since these data types are optimized to hold those values, therefore saving storage and performing better.

A key takeaway when it comes to optimizing data types is understanding your data. Not only by column name, but the values that are included in the columns. By understanding them, you'll be able to choose the best data type for that column.

## Optimize Your Queries

In addition to reviewing your database and data types within it, optimizing your queries will also be helpful in optimizing the overall performance of your database. Part of optimizing your queries entails using the **EXPLAIN** statement to see your query's execution plan and reviewing your tables' indexes to ensure that they are used appropriately.

For a refresher on how you can optimize your queries, take a look at the [Improving Performance of Slow Queries in MySQL Reading!](#)

## Next Steps

Now that you're more familiar with how you can monitor and optimize your query, let's take a look at how we can do this in practice with phpMyAdmin in the Skills Network Labs environment!

## Author(s)

Kathy An

## Other Contributor(s)

## Changelog

| <b>Date</b> | <b>Version</b> | <b>Changed by</b>          | <b>Change Description</b> |
|-------------|----------------|----------------------------|---------------------------|
| 2023-05-11  | 1.2            | Eric Hao & Vladislav Boyko | Updated Page Frames       |
| 2021-10-12  | 1.0            | Kathy An                   | Created initial version   |
| 2021-10-13  | 1.1            | Steve Hord                 | Copy edits                |

© IBM Corporation 2023. All rights reserved.